

masCon / next.app devCon · Berlin 2026

Security Risks of Packages in Mobile App Development

Tsuyoshi Chujo Flutter developer · package author [@tsuyoshi_chujo](#)

About me



Tsuyoshi Chujo

Flutter developer from Japan

Package user

Cardcraft, and more

Package author

`crop_your_image`, `animated_to`, and more

~~Security expert~~

606

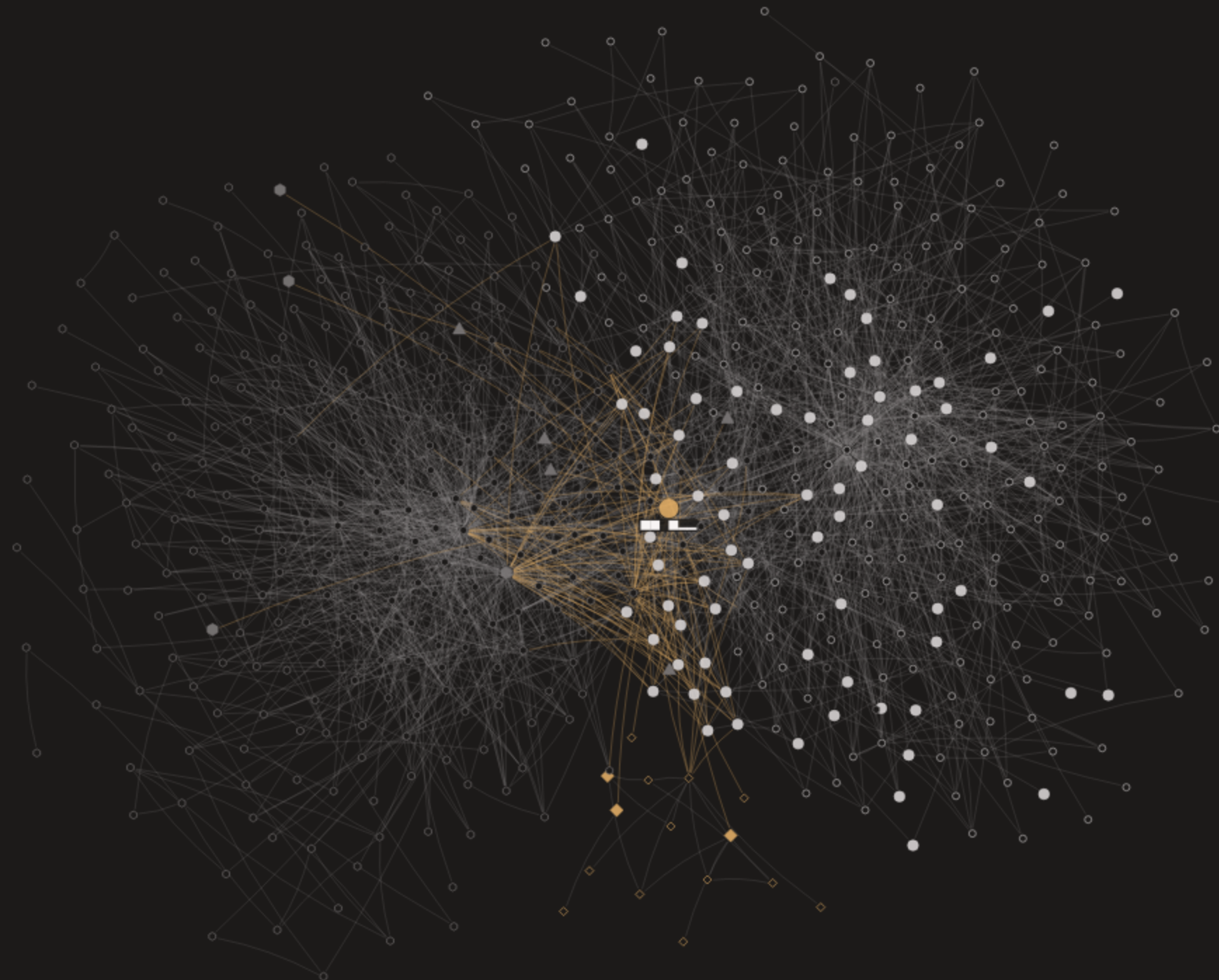
Packages my production Flutter app depends on.

Dart 283

iOS 15

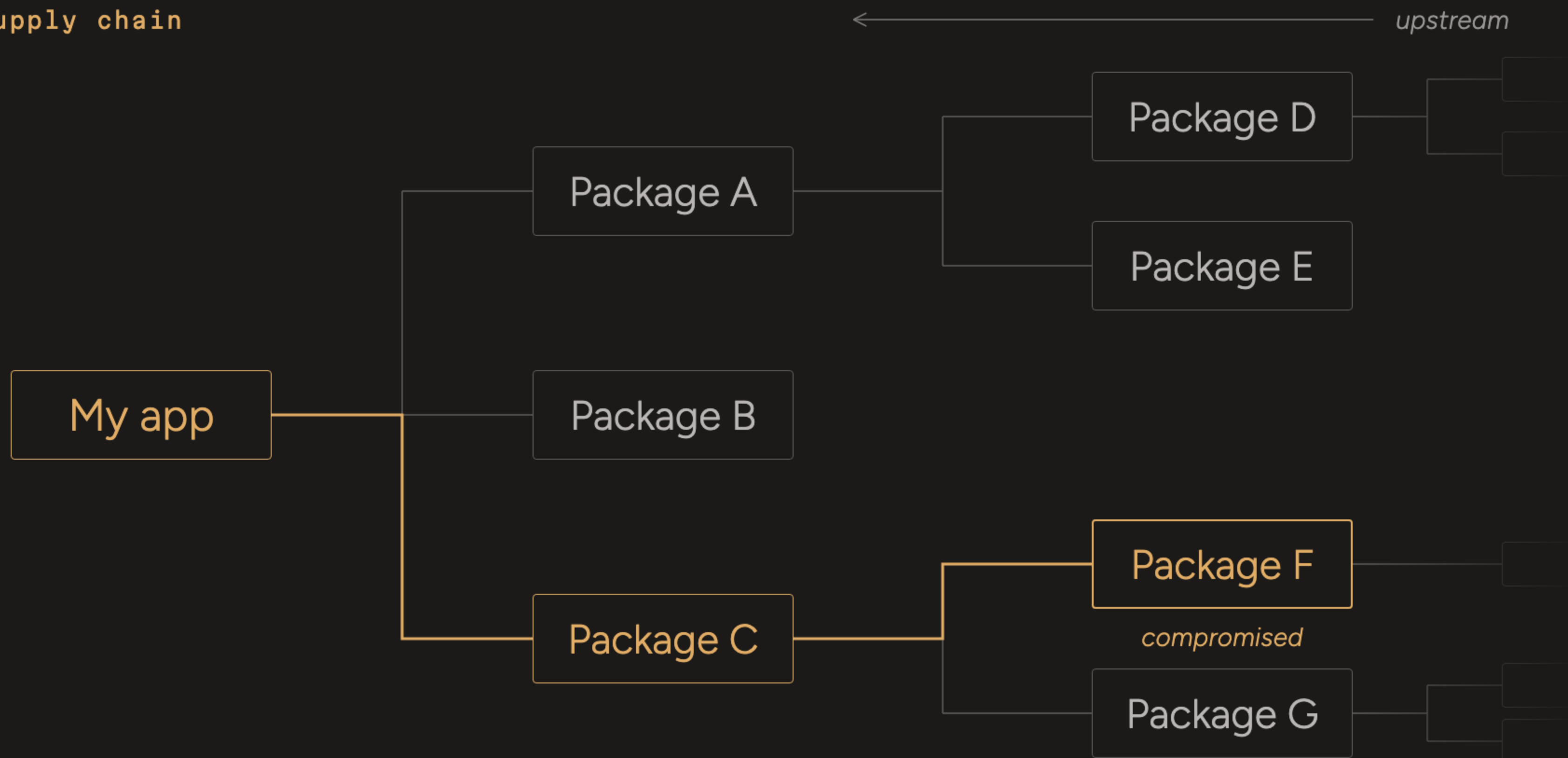
Android 308

606 dependencies, direct and transitive



● Dart 283 ● iOS 15 ● Android 308 Filled = direct · Outline = transitive

Supply chain



Supply chain

Impact examples

DEVELOPERS / CI

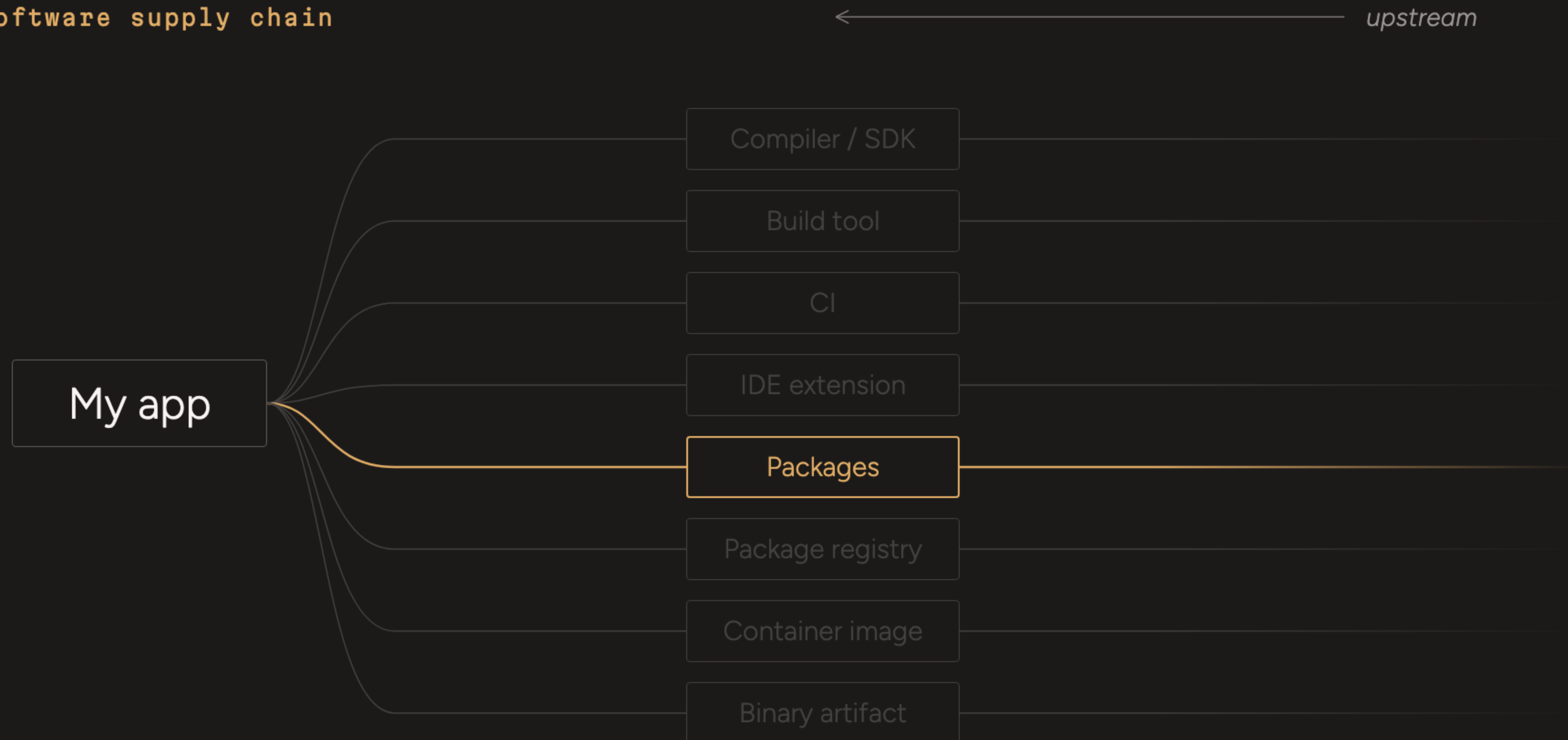
- Credential theft
- Private source code leaks
- Malicious versions published with stolen credentials

END USERS

Malicious code ships inside the app.

The damage is anything the app can do on the device — too varied to enumerate.

Software supply chain



Real-world incidents

npm · 2026

Red Hat packages

A malicious preinstall hook was added to legitimate packages

npm install alone executed it on dev machines and CI

access.redhat.com

PyPI · 2026

LiteLLM / Telnyx

Publish credentials compromised; a version with a credential stealer published

Stolen tokens were used to attack further packages

blog.pypi.org

pub.dev · 2026

universal_file_viewer

XCSSET on the maintainer's Mac infected the Android / Xcode projects in example/

Normal commit and publish carried it to GitHub and pub.dev — the package was not the target

aikido.dev

MALICIOUS CODE

exists $\neq$ *executes*

When can a package actually execute code?

Execution surfaces

When does package code actually run?

MECHANISM	WHEN	EXPLICIT OPT-IN?
Library code	at runtime	— you call it
Build hooks / link hooks	during the build	No part of the normal build
Analyzer plugins	during analysis	Yes explicit configuration
build_runner / builders	during code generation	Yes you run build_runner
DevTools extensions	while debugging	Yes user enables it

Build hooks

deep dive

Build hooks

What build hooks do

- Native code compilation
- Native asset preparation
- Prebuilt native asset download
- Linking information preparation
- Platform-specific build integration



What real hooks do

03 / 03

Search the developer machine

android_libc++_shared discovers a locally installed Android NDK and bundles its libc++_shared.so — no download, no compile

android_libc++_shared 0.3.0 

hook/build.dart

```
// hook/build.dart - excerpt
final resolution =
    await resolveLibc++_Shared(input, logger:
logger);
// searches compiler paths, PATH, ANDROID_NDK_HOME,
// local.properties, common install locations, ...
output.assets.code.add(CodeAsset(
    name: 'libc++_shared.so',
    file: resolution.libc++_Shared,
    linkMode: DynamicLoadingBundled(),
));
```

A new version can ship a new hook

some_package
1.4.2

some_package
1.5.0

hook/build.dart

changed · runs on your next build

pubspec.lock
some_package: 1.4.2

The version you resolved is the version you build with.

Nothing in the package changed

Yesterday

package 1.4.2

hook/build.dart

tool.zip

Today

package 1.4.2

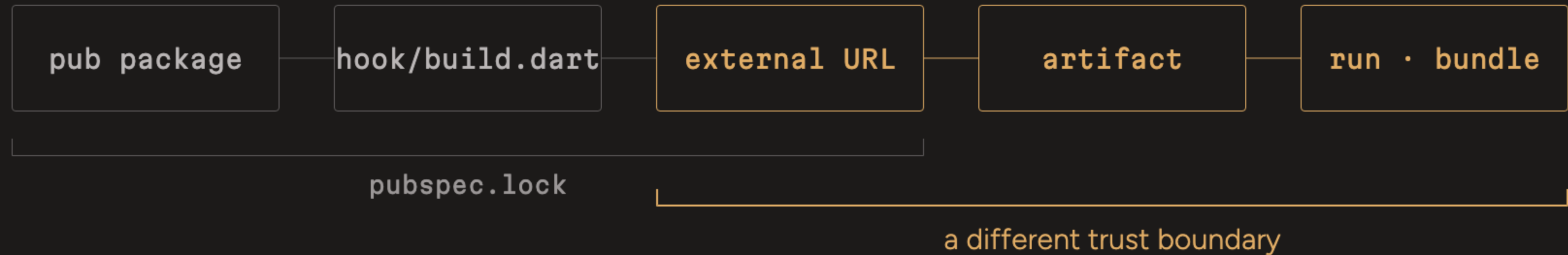
hook/build.dart

tool.zip
replaced upstream

Can this dependency change its behavior without changing itself?

Build hooks · trust boundary

Same version. Same bytes?



Is the source on GitHub
what my build actually uses?

No.

Audit what your build actually consumes.

provenance

Inspect

Get the exact packages your build runs

~/ .pub-cache

Hosted packages your build resolved are already on disk

dart pub unpack

Fetches any package version into a local directory for inspection

```
$ dart pub unpack some_package
```

The exact set of packages that will run is always available to read.

Beyond build hooks

Ask the same questions elsewhere

Analyzer plugins

When does it run?

Explicit opt-in?

What does it fetch or run?

What does the build consume?

build_runner / builders

When does it run?

Explicit opt-in?

What does it fetch or run?

What does the build consume?

DevTools extensions

When does it run?

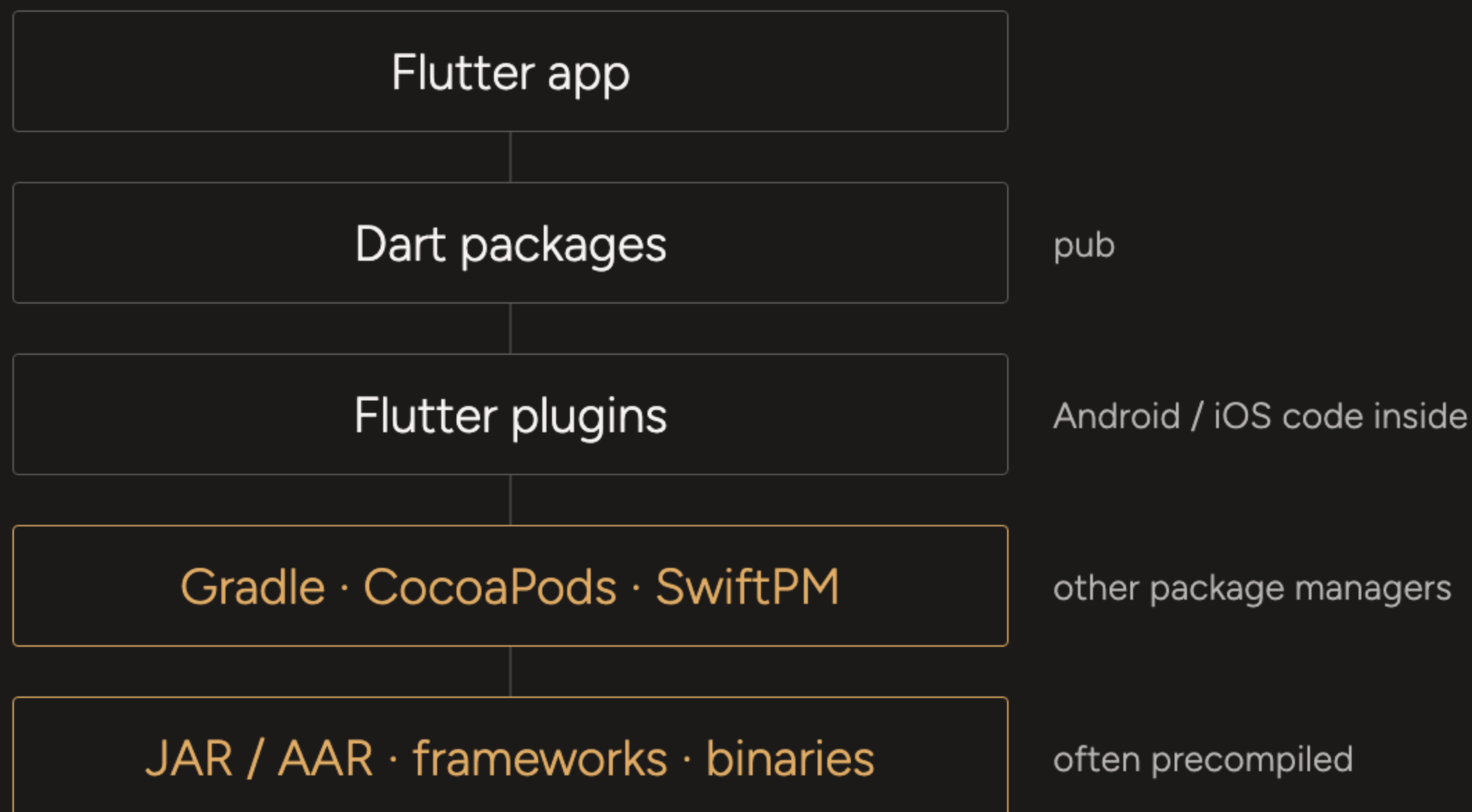
Explicit opt-in?

What does it fetch or run?

What does the build consume?

Beyond Dart

A Flutter app depends on more than Dart



Provenance

Source distribution vs binary distribution

ECOSYSTEM / MECHANISM	TYPICAL FORM	BUILD INPUT
Dart / pub	source package	source
CocoaPods source_files	source	source
SwiftPM .target	source	source
Maven / Gradle JAR / AAR	precompiled	binary artifact
CocoaPods vendored framework	precompiled	framework / XCFramework
SwiftPM .binaryTarget	precompiled	binary artifact
DevTools extension	precompiled web output	compiled extension artifact

Is this the source that produced the binary I actually consume?

Cooldown — wait before you update

Adopt a new version only after it has been public for a while.

e.g. Dependabot cooldown · Renovate `minimumReleaseAge`

COVERS	Many compromised releases are detected and pulled within days — a cooldown skips exactly that window
DOES NOT COVER	Malware that stays unnoticed longer than your cooldown — and it does not protect whoever updates first
TRADE-OFF	Legitimate fixes, including security patches, also arrive late

Respect the lockfile

Build from exactly what pubspec.lock records — in CI too.

```
$ dart pub get --enforce-lockfile
```

COVERS	No unintended version enters the build — every update becomes an explicit, reviewable diff
DOES NOT COVER	A locked version that is already malicious — and what the lockfile does not pin, like the artifact a hook downloads
TRADE-OFF	Updates need deliberate maintenance — falling behind accumulates unpatched issues

Mitigations

No single “supply chain protection” checkbox

CONTROL	COVERS	DOES NOT COVER
Lockfile	unexpected updates	malicious locked content
Pub content hash	the package archive	what the hook fetches later
Source review	source behavior	every dependency, every time
Scanners	known bad patterns	malicious logic, reliably
Sandboxes	what crosses the boundary	everything inside the boundary
Provenance	where the bytes came from	safety of the source
Binary analysis	the artifact itself	costs — far more than source review

606

Can we do this for all 606 dependencies?

No. Understand the mechanisms, then prioritize.

AI makes going one level deeper much cheaper — the questions are still yours.

01 Understand your platform

If you don't know the mechanism, you don't know what you trust.

02 Understand the attack path

Where it enters, where it executes, who is affected.

03 Choose mitigations based on that understanding

Pick the control that stops a concrete attack path.

Thank you.

Tsuyoshi Chujo @tsuyoshi_chujo

Go one level deeper.